

CS6004: Code Optimisation for OO Languages

PA4: Beat the Interpreter

Group Name: Zero Overhead

Members: (1) Ketan Patil - 25M0788

(2) Nikhil Gour - 25M0789

Optimization Description

What is Implemented?

The optimisation we have implemented is **monomorphisation**, along with the use of polymorphic inline caching (PIC).

At a high level, when a virtual call site is determined, using analysis, to have only a single possible concrete method, we replace this virtual call with an equivalent static call. If the callsite has multiple possible targets, but the number of targets is small, we transform it into a sequence of conditional checks or into an if-else ladder (PIC implementation). Each branch checks the runtime type receiver and invokes the corresponding static call. The information required to determine these targets comes from the object-sensitive points-to analysis, through which we get the precise receiver information at each call site.

For each such transformation, we are generating a static stub. It is a copy of the original method body where the 'this' pointer reference is converted into an explicit parameter. Again, this is necessary as static methods do not have 'this' references. The original callsite is also rewritten to invoke the static stub instead of making a virtual call.

Additionally, the transformation also works recursively. So, if the newly created static stubs contain the virtual call, that too can be resolved and transformed.

Why is it useful?

In object-oriented languages like Java, most of the method calls are resolved at runtime through virtual dispatch, which involves an extra level of indirection (vtable lookup), which adds overhead. This also limits low-level optimisation, such as inlining and constant propagation.

Monomorphisation helps in resolving many of the call sites at compile time by using the point-to analysis. When the call site is reduced to a single call target, then it can be converted to a direct static call, which eliminates the overhead of dispatching. Even in some cases where multiple targets exist, PIC avoids the full virtual dispatch by replacing it with a small number of type checks, which are followed by direct calls.

The use of static stubs further improves the optimisation opportunities in the program, as the calls become explicit, it can be utilise of inlining and further specifications.

Analysis Used

Object-sensitive points-to analysis

We have used the k-level **object-sensitive points-to analysis**. Here, the goal of this analysis is to determine the possible set of receiver objects at each site. Once we get to know which objects a variable points to, we can infer which method might be invoked at that site.

We have analysed each method along with the context, where the context of a method is a sequence of the last k receiver objects. So, instead of analysing a method once, we are analysing the method separately with the contexts.

This approach is more precise than the standards call-graph algorithms of VTA and RTA, as it utilises the allocated object information rather than just tracking the types.

Precision Dimensions

- **Flow-insensitivity:** The statements in the body are processed without ordering, but this may introduce imprecision as different program paths can merge together.
- **Context-sensitivity (k level):** Each method is analysed once per unique k-object calling context, not just once per method.
- **Field Sensitivity:** The analysis is field-sensitive for the instance fields, where each allocation object maintains a separate mapping of its field.
- **Interprocedural Analysis:** The analysis is interprocedural, where the points-to information is maintained across the program.

Assumptions & Limitations

- We only analyse concrete methods.
- Array elements are not precisely tracked.
- Library classes are ignored.
- Here, static fields are context-insensitive.
- Analysis may lose precision for deeply nested object interactions.
- No strong updates are performed here due to flow insensitivity.

Transformational details

Monomorphic case

When a call site resolves to exactly one context-method target, the transformation proceeds as follows:

Before (Jimple): virtual call through a base reference

```
$r1 = new Circle;
specialinvoke $r1.<Circle: void <init>()>();
$r2 = virtualinvoke $r1.<Shape: double area()>();
```

After (Jimple): static call to a generated stub

```
$r1 = new Circle;
specialinvoke $r1.<Circle: void <init>()>();
$r2 = staticinvoke <Circle: double stub_area_c_r1newCircle(Circle)>($r1);

class Circle extends Shape
{
    public static void stub_area_c_r1newCircle(Circle)
    {
        ...
        r0_explicit := @parameter0: Circle;
        ...
    }
}
```

The stub here itself is a copy of the Circle.area() method, where the implicit receiver is replaced by the explicit Circle parameter. This means there will not be a need for vtable lookups.

PIC case

When more than 1 but less than 3 targets exist, we transform it into conditional checks having a specific static call:

Before (Jimple)

```
$r2 = $r1.<Wrapper: Base f>;
virtualinvoke $r2.<Base: void foo()>();
```

After (Jimple)

```
$pic_cond = $r2 instanceof B;
if $pic_cond == 0 goto label1;
$pic_cast_1 = (B) $r2;
staticinvoke <B: void stub_foo_c_r4newB1(B)>($pic_cast_1);
goto label2;
label1:
$pic_cast_2 = (Base) $r2;
staticinvoke <Base: void stub_foo_c_r2newA1(Base)>($pic_cast_2);
goto label2;
label2:
return;
```

Here, the branches are sorted in the most specific order, subclass before superclass, which avoids the false matching of instances.

Key Challenges

- Cloning the method bodies while generating new static while preserving the actual control flow of the method.
- While new stub generation, managing the references of 'this' to our explicit local variable.
- Recursive transformations in the program, if the newly created stub has virtual calls.
- While building PIC, we need to ensure that the more specific type (subclass) is checked before the more general type (superclass).
- Design choice for building object-sensitive points-to analysis, while preserving the precision.

Results

Functional Correctness

All the test cases produce the same output before and after transformation. This tells that the optimisation preserves program semantics.

Performance Results

Testcase	Before optimisation (ms)	After optimisation (ms)	Improvement (%)	Contexts Formed	StaticStubs generated
Test1	38.132	37.539	1.56	6	1
Test2	50.080	50.542	-0.92	14	0
Test3	98.730	92.566	6.24	8	3
Test4	97.472	100.133	-2.73	10	2
Test5	81.060	73.840	8.91	23	0
Test6	72.973	71.356	2.22	12	3
Test7	41.783	41.763	0.05	28	3
Test8	75.977	74.139	2.42	12	3
Test9	63.438	55.640	12.29	6	2

Testcase	Before optimisation (ms)	After optimisation (ms)	Improvement (%)	Contexts Formed	StaticStubs generated
Test10	87.929	92.083	-4.72	18	4

Observations

- The optimisation shows a clear improvement in the test cases, where virtual calls resolved to single targets. Test1, Test2, and Test9 demonstrate the monomorphic pattern where the receiver always points to the concrete class.
- Recursive and repeated call scenarios benefit more from the optimisation, as the same virtual call is executed many times by the recursion; our optimisation provides an overall performance benefit.
- The PIC-based transformations, as in Test4 and Test10, provide very limited or negative improvement, as typecheck overhead might outweigh the transformation improvement.
- In Test7, which includes monomorphic, PIC, and fully virtual call sites together, shows very low improvement (0.05%). This indicates that gains from optimisation can be compensated for by overheads in other parts of the program.